

M12 — Activity catalog: routes + wizard integration

Phase: 7 — Activities **Status:** DONE (2026-05-26) **Blocked by:** M11 **Blocks:** M14 **Estimated effort:** 2 days · **Actual:** 0.5 day

1. Purpose

Take M11's data layer live. M12 ships:

1. **HTTP CRUD surface** at `/api/activities` — GET list, GET one, POST create (custom only), PATCH update, DELETE, plus a convenience GET `/api/activities/{id}/encounter_entry` that pre-builds the wizard's encounter-row payload. 2. **Wizard integration** — the room-mode Step 4r editor (M6) gains an Activity picker column. Picking an activity pre-fills the row's label + persona; on submit the row carries the activity's `scenario_text`, `seed_modules` (union with wizard-wide), and `default_chart_mode`. 3. **Server-startup seed hook** — `activities.seed_builtins()` runs on every server boot via FastAPI's `@app.on_event("startup")`. Idempotent.

2. Structure

Files touched:

- `portal/server.py` —
- Imports `activities`.
- Adds the `_seed_activity_catalog` startup hook.
- Appends the M12 "V7 — Activity catalog HTTP API" section with 6 new routes. - Updates the wizard's `control_wizard` context to inject `activities_for_room` from `ehr_db.list_activities()`.
- `portal/templates/control.html` — adds `activitiesForRoom` to `window.MEDSIM2`.
- `portal/static/control.js` —
- `renderRoomEncounterRows` adds a 4-option `<select>` column (the Activity picker) in each row. - New global change handler — when an Activity is picked, stashes `data-scenario-text`, `data-activity-id`, `data-chart-mode`, `data-seed-modules-json` on the row and pre-fills label + persona. - `submitRoom` reads the stashed fields and merges them into the per-encounter POST body.
- `portal/static/control.css` — `encounter-row` grid template goes from 4 to 5 columns.

No new module files — M12 is glue: routes + JS + CSS only. The Activity data model lives in `portal/activities.py` (M11).

3. Uses

- **Wizard:** the operator picks an Activity in Step 4r → label

+ persona pre-fill, then can adjust per-row → on Start, the room's encounters carry `activity_id` and the activity-derived `scenario_text` / `modules` / `chart_mode`.

- **Future Author/Edit UI:** a follow-up dashboard surface

(post-MVP, deferred) will POST/PATCH/DELETE activities. Until then operators can `curl` against the routes or run a small Python script.

- **M14 cohort debrief** (next module up) reads

`activity.answer_key` for rubric-based scoring of student documentation.

4. Functions (exported API surface)

HTTP routes

Method	Path	Auth	Body / Query	Returns
GET	<code>/api/activities</code>	operator vault	<code>?builtin_only=true</code> (optional)	<code>{activities: [<rows>]}</code> (built-ins first, alphabetical)
GET	<code>/api/activities/{id}</code>	operator vault	—	row dict or 404
POST	<code>/api/activities</code>	operator vault	<code>{label*, seed_persona_id?, seed_modules?, scenario_text?, default_chart_mode?, answer_key?}</code>	persisted row; <code>is_builtin</code> is always False on this path
PATCH	<code>/api/activities/{id}</code>	operator vault	subset of fields (unknown fields silently ignored)	updated row or 404
DELETE	<code>/api/activities/{id}</code>	operator vault	—	200 (idempotent), 409 if the row is built-in
GET	<code>/api/activities/{id}/encounter_entry</code>	operator vault	—	wizard-row dict matching <code>/api/room/start</code> body shape, or 404

Startup hook

`_seed_activity_catalog()` — runs once at FastAPI startup. Wraps `activities.seed_builtins()` in a try/except so a seed failure never blocks the server from booting (warning logged to stderr).

Wizard JS additions

- `window.MEDSIM2.activitiesForRoom` — list of activity rows

injected by the template at page load.

- Per-row data-`*` stash on each `.encounter`-row:
- `data-scenario-text` — the activity's scenario text
- `data-activity-id` — activity id (carried into the POST body)
- `data-chart-mode` — `shared` or `private_clone` per the activity

- `data-seed-modules-json` — activity modules, merged into the row's modules at submit time
- `submitRoom` merges these into the per-row encounter object sent to `/api/room/start`.

5. Limitations

- **No CRUD UI for non-wizard surfaces.** Authoring + editing activities outside the wizard requires HTTP (curl, postman, or a follow-up dashboard route). M12 is the `data+-routes+-wizard` layer; an admin UI is deferred.
- **No CSV import / export.** Bulk catalog management is a v7.1 candidate.
- **PATCH whitelist matches M11.** Editable fields: `label`, `seed_persona_id`, `seed_modules`, `scenario_text`, `default_chart_mode`, `answer_key`. To add a field, extend the whitelist in `ehr_db.update_activity`.
- **DELETE on built-in returns 409.** No "force delete" route. Resetting a built-in to its catalog version is left to M13+.
- **Wizard picker doesn't show activity metadata** beyond the label. A v7.1 enhancement would surface persona, modules, and a 1-line summary on focus.
- ****Activity picker pre-fills label only if the existing label starts with "Bed N".**** Operator-edited labels are respected. This is a heuristic — a future "Reset row" affordance would make the behavior explicit.
- ****The `_seed_activity_catalog` startup hook uses `@app.on_event("startup")` which FastAPI marks as deprecated in favor of lifespan events. The deprecation is non-fatal; switching to lifespan is a small follow-up that should batch with any other `on_event` uses in the codebase.**

6. Test status

Automated (`tests/v7/test_api_activities_crud.py`, `test_wizard_picking_activity_seeds_encounter.py`)

Test file	Cases	Status	Last run
test_api_activities_crud.py	12 — list returns 8 built-ins, builtin_only filter, GET 404s, GET round-trip, POST create custom, POST rejects invalid chart_mode / blank label, PATCH updates, PATCH 404s, DELETE refuses built-ins, DELETE custom, DELETE idempotent, encounter_entry round-trip.	PASS	2026-05-26
test_wizard_picking_activity_seeds_encounter.py	4 — encounter_entry → /api/room/start round-trip carries activity_id + modules + persona + scenario_text + chart_mode; mixed activity + custom row in one room; custom activity round-trip; startup seed hook ran.	PASS	2026-05-26

17/17 PASS. **Full v7 suite: 84/84 passing** (up from 67 — +17 M12 tests). **Full v6 regression on v7: 195 passed**, same 6 env-flaky pre-existing failures, **0 v7 regressions**.

Manual

Browser preview verification deferred — the wizard's Activity picker is a routine select-box addition; the underlying contract (activity picked → finalize → encounter carries the activity's fields) is exercised end-to-end by test_wizard_picking_activity_seeds_encounter.py. M20 Playwright will exercise the click path.

7. Change list

Date	Author	Change	Files
2026-05-26	claude-cod e	6 new routes (/api/activities GET list + GET one + POST + PATCH + DELETE + /encounter_entry). FastAPI startup hook _seed _activity_catalog. Wizard JS adds an Activity picker column per encounter row; submit merges activity-derived fields. CSS grid bumped from 4 → 5 columns. 17 acceptance tests across 2 files.	portal/server.py, portal/templates/control.html, portal/static/control.{js,css}, tests/v7/test_api_activities_crud.py, tests/v7/test_wizard_picking_activity_seeds_encounter.py

8. Open questions / known issues

- `@app.on_event("startup")` is deprecated in FastAPI ≥ 0.93 .

Lifespan events are preferred. Migration is one-line; deferred to a hygiene-pass module so it batches with any other `on_event` uses.

- The Activity picker reads from a snapshot of the catalog at

page load. If an admin creates a new activity in a separate tab, the wizard won't see it until refresh. A 30-second poll or a "Refresh" button could help; not urgent at the M12 scale.

- Mixing activity rows + custom rows in one room (covered by

`test_wizard_picking_activity_in_one_of_many_rows`) works end-to-end. The activity-driven `scenario_text` overrides the Step 3 free-form text on a per-row basis. If the operator wants the Step 3 text as a baseline for an activity-seeded row too, they have to either edit the activity to embed it OR copy-paste manually. Worth a UX revisit when M14 debrief lands and we see how often this comes up.

- The deprecated `@app.on_event` use surfaces in test logs as a

`DeprecationWarning`. Test infrastructure already filters most warnings; left visible for now so the migration item stays on the radar.